

How to store and route your (physical) mail like a pro personal edition

The usual view after vacation



Bart Dorlandt



- Freelance Network Automation Solution Architect
- 20 years in network engineering and automation
- Last year I presented "Repos are like children - Parenting 101"
- Lives by the phrase: *There must be a better way*



Pragmatic approach to a better mail handling

- The problem: physical mail is a mess
- The goal: have a better way to handle it, store it, and find it
- The solution: a mail handling system that is flexible, automated, and easy to use

Paperless-ngx

- I had heard about paperless-ngx, an open-source document management system
- I have a few friends that were already happy with it

What is paperless-ngx

- A document management system that allows you to store, organize, and search your documents
- It has a web interface, an API, and can read and parse emails
- It can also watch folders for new documents to process
- It has features for structuring, self-improving & workflows

Paperless-ngx

Search documents

Dashboard

Documents

SAVED VIEWS

Inbox

Recently Added

MANAGE

Correspondents

Tags

Document Types

Storage Paths

Custom Fields

Templates

Mail

ADMINISTRATION

Settings

Users & Groups

File Tasks 16

Logs

Documentation

Paperless-ngx v2.0.0

Dashboard

Hello User2, welcome to Paperless-ngx

Inbox

Show all

Created	Title	Tags	Correspondent
Aug 9, 2023	H7_Napoleon_Bonaparte_zadanie	Another Sample TagInbox	Newest Correspondent
Mar 25, 2023	[paperless] test post-owner	InboxTag 2	Test Correspondent 1
Oct 2, 2022	1 Testing New Title Updated 2	Another Sample TagInboxTagWithPartial	Correspondent 9
Oct 2, 2022	Sample100.csv	InboxJust another tagTag 2	Newest Correspondent
Sep 11, 2022	drylab Test	InboxTest Tag	

Recently Added

Show all

Created	Title	Tags	Correspondent
Aug 9, 2023	H7_Napoleon_Bonaparte_zadanie	Another Sample TagInbox	Newest Correspondent
May 30, 2023	0004814539_20230531		
Mar 25, 2023	[paperless] test post-owner	InboxTag 2	Test Correspondent 1
Dec 11, 2022	tablerates2	TagWithPartial	
Jul 19, 2021	sat-practice-test-1_pt2-collated	NewOneTag 12	
Jun 30, 2018	[paperless] Owned by Test Not Shared	Another Sample TagTag 2	

Paperless

ngx

Statistics

Documents in inbox:5

Total documents:64

Total characters:2,514,649

PDF (93.8%)TXT (3.1%)CSV (3.1%)

Tags:28

Correspondents:7

Document Types:5

Storage Paths:4

Upload new documents

Drop documents anywhere or

Browse files

7

Paperless-ngx

Search documents

Dashboard

Documents

SAVED VIEWS

Inbox

Recently Added

OPEN DOCUMENTS

2sample-pdf-with-images

H7_Napoleon_Bonaparte_zadanie

Close all

MANAGE

Correspondents

Tags

Document Types

Storage Paths

Custom Fields

Templates

Mail

ADMINISTRATION

Settings

Users & Groups

File Tasks16

Logs

Documentation

Paperless-ngx v2.0.0

2sample-pdf-with-images

DiscardSave & nextSave

DetailsContentMetadataNotes1Permissions

Title2sample-pdf-with-images

Archive serial number+1

Date created03/27/2022

Correspondent+

Document typeAnother Type+

Storage pathTesting 12+

Tags

Another Sample Tag

+

Suggestions: TagWithPartial

DiscardSave & nextSave

Page1of 10

Delete

Download

Actions

Custom Fields

Share Links

Boundary Waters Trip

Six days in BWCA

Legend

Day 1

Day 2

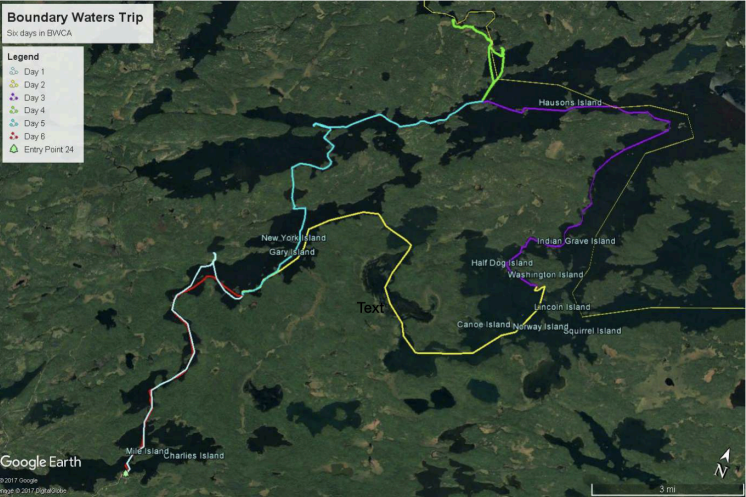
Day 3

Day 4

Day 5

Day 6

Entry Point 24



Google Earth

Curabitur bibendum ante uma, sed blandit libero egestas id. Pellentesque rhoncus elit in lacus ultrices fringilla. Nam ac metus eu turpis mattis rutrum. Mauris mattis sem ex, facilisis molestie sapien luctus non. Vestibulum tincidunt uma at odio suscipit, vel congue felis cursus. Etiam tellus magna, egestas ac suscipit in, laoreet quis felis. Proin non orci id dui tincidunt egestas.

Vestibulum eleifend, ligula a scelerisque vehicula, risus justo ultricies ligula, et interdum lorem ex eget ex. Duis dignissim lacus vitae velit laoreet, vitae placerat velit aliquet. Etiam eget mollis nulla, ac vehicula mi. Etiam non sollicitudin velit, imperdiet commodo mi. Fusce quis tellus tellus. Donec dictum euismod risus non tempus. Duis quis pellentesque nunc. Praesent elementum condimentum mollis.

Phasellus dapibus quam a hendrerit placerat. Sed ultrices blandit nulla sed sodales. Nunc quis volutpat eros. Etiam bibendum eu tellus consequat blandit. Curabitur lacinia cursus diam sed pharetra. Proin molestie tristique mauris ut aliquam. Donec purus odio, molestie id suscipit sit amet, porttitor in erat. Vestibulum ut tellus vel mi lobortis porta nec vel tellus. Quisque pretium blandit dignissim. Proin eu metus convallis sapien efficitur mollis. Nunc luctus ex in nunc ornare, nec blandit orci faucibus. Aenean bibendum mi vel neque euismod hendrerit. Vestibulum ac pharetra magna. Ut rutrum, orci at blandit faucibus, justo mauris iaculis mauris, ut tempor lectus risus at ligula.

Duis non tincidunt purus. Nam quis sapien risus. Donec mattis convallis tempor. Fusce aliquam aliquet eros, nec rutrum lectus pretium at. Praesent blandit justo a mi dignissim placerat. Ut ullamcorper elit eget diam maximus iaculis. In bibendum in massa eget facilisis. In iaculis lectus

9

Paperless-ngx

```
services:
  broker:
    image: docker.io/library/redis:7
    restart: unless-stopped
    volumes:
      - redisdata:/data

  webserver:
    image: ghcr.io/paperless-ngx/paperless-ngx:latest
    restart: unless-stopped
    depends_on:
      - db
      - broker
      - gotenberg
      - tika
    ports:
      - "8078:8000"
    volumes:
      - data:/usr/src/paperless/data
      - /.../media:/usr/src/paperless/media
      - /.../export:/usr/src/paperless/export
      - /.../consume:/usr/src/paperless/consume
    env_file: docker-compose.env
    environment:
      PAPERLESS_REDIS: redis://broker:6379
      PAPERLESS_DBHOST: db
      PAPERLESS_TIKA_ENABLED: 1
      PAPERLESS_TIKA_GOTENBERG_ENDPOINT: http://gotenberg:3000
      PAPERLESS_TIKA_ENDPOINT: http://tika:9998
```

```
db:
  image: docker.io/library/postgres:16
  restart: unless-stopped
  volumes:
    - pgdata:/var/lib/postgresql/data
  environment:
    POSTGRES_DB: paperless
    POSTGRES_USER: paperless
    POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"

gotenberg:
  image: docker.io/gotenberg/gotenberg:8.7
  restart: unless-stopped

  # The gotenberg chromium route is used to convert .eml files. We do not
  # want to allow external content like tracking pixels or even javascript.
  command:
    - "gotenberg"
    - "--chromium-disable-javascript=true"
    - "--chromium-allow-list=file:///tmp/*"

tika:
  image: docker.io/apache/tika:latest
  restart: unless-stopped

volumes:
  data:
  pgdata:
  redisdata:
```

Destinations

Where do my documents need to go?

- Paperless
- Bookkeeping system (email / app / web)
- Bookkeeper (email)

Input

- I don't want to use this
- I "can't" use the camera app on the phone
 - doesn't work well in the dark
 - doesn't generate a pdf
- It should be easy!



Dropbox camera

- It takes the picture, and:
 - enhances the contrast
 - finds the corners
 - generates a pdf
 - uploads it to the dropbox cloud in a chosen folder

"NOTE: you'll have to accept this is synced to the cloud."

Fahrzeughaus Laufer		
Kesselstr. 1		
79843 Loeffingen		
Freie Tankstelle		
07654 / 7137		
- StNr. Station: 13185-10202 -		
Bon-Nr.:	521204	Datum: 06.05.2026
Verk. :	1	Zeit: 09:52:35
Menge	Einzelpreis	Gesamtpreis
* Diesel 1		Säule 1 ST:D *
* 38,06 Liter	2,019 EUR	76,84 EUR*
*	pro Liter	*
GESAMTSUMME:		76,84 EUR
-K-U-N-D-E-N-B-E-L-E-G-		
Terminal-ID :	75087016	
TA-Nr 212984	BNr 0122	
Kartenzahlung		
kontaktlos		
DEBIT MASTERCARD		
EUR 76,84		

Solution dependencies

- Synced folder
- Some code to glue it all together
- Execute the code on a regular basis
- Paperless

What do we need?

- CloudSync to sync the dropbox folder to the NAS
- Docker + docker-compose
- Python
- Task Scheduler

Requirements of this little glueware

- Ability to send to multiple destinations
- Ability to process folders for new files
- Ability to move files to done folder
- Ability to notify me in case of errors

Let's dive in

```
def main() -> None:
    """Main entry point for the script."""
    # Reading the vars here to ensure env dependencies are present and loaded
    paperless_vars = PaperlessVars(
        api_token=env.str("PAPERLESS_API_TOKEN"),
        api_path=env.str("PAPERLESS_API_PATH"),
        api_url=env.str("PAPERLESS_API_URL"),
    )
    bookkeeping_vars = EmailVars(
        to=env.str("BOOKKEEPING_EMAIL", validate=[validate.Length(min=4), validate.Email()]),
    )
    bookkeeper_vars = EmailVars(
        to=env.str("BOOKKEEPER_EMAIL", validate=[validate.Length(min=4), validate.Email()]),
    )
    ...

if __name__ == "__main__":
    MAIN_PATH = Path(env.str("PROCESS_FOLDER"))
    SMTP_VARS = SmtVars(
        smtp_srv=env.str("SMTP_SRV"),
        smtp_usr=env.str("SMTP_USR", validate=[validate.Length(min=4), validate.Email()]),
        smtp_pwd=env.str("SMTP_PWD"),
        smtp_port=env.int("SMTP_PORT", default=465),
    )
    ERROR_EMAIL = env.str("ERROR_EMAIL", validate=[validate.Length(min=4), validate.Email()])
    FROM_EMAIL = env.str("SMTP_USR", validate=[validate.Length(min=4), validate.Email()])
    main()
```

```
@dataclass
class SmtVars:
    """Email (SMTP) configuration."""

    smtp_srv: str
    smtp_usr: str
    smtp_pwd: str
    smtp_port: int = 465

@dataclass
class PaperlessVars:
    """Paperless API configuration."""

    api_token: str
    api_path: str
    api_url: str

@dataclass
class EmailVars:
    """Email configuration."""

    to: str
```

```
def main() -> None:
    # 2nd part of the main function, after reading the vars
    paperless_processor = PaperlessAPIProcessor(paperless_vars)
    bookkeeping_processor = EmailProcessor(bookkeeping_vars)
    to_person_processor = EmailProcessor(bookkeeper_vars)

    logger.info("Loaded variables, processing directories...")
    process_folder(MAIN_PATH / "to_paperless", processors=[paperless_processor])
    process_folder(MAIN_PATH / "to_bookkeeping", processors=[bookkeeping_processor])
    process_folder(MAIN_PATH / "to_bookkeeping_paperless", processors=[paperless_processor, bookkeeping_processor])
    process_folder(MAIN_PATH / "to_paperless_bookkeeper", processors=[paperless_processor, to_person_processor])
    process_folder(MAIN_PATH / "to_bookkeeper", processors=[to_person_processor])
```

```

def process_folder(folder: Path, processors: list[FileProcessor]) -> None:
    """Process files in the given folder with the provided processors.
    Files starting with a dot are ignored.
    """
    folder.mkdir(exist_ok=True)
    for file in list(folder.iterdir()):
        if not file.is_file() or file.name.startswith("."):
            continue

        # Track if any processor succeeded
        processed = 0
        for processor in processors:
            # Only move to done if all processors succeed
            result = processor.process(file)
            processed += result
            if not result:
                error_email(subject="File Processing Error", filename=file.name)

    if processed == len(processors):
        move_to_done(file)

```

```

def move_to_done(filepath: Path) -> None:
    """Move processed file to the done directory."""
    parent = filepath.parent.name
    done_dir = MAIN_PATH / "done" / parent
    done_dir.mkdir(parents=True, exist_ok=True)
    target = done_dir / filepath.name
    filepath.rename(target)
    logger.info(f"Moved {filepath} to {target}")

```

```
from typing import Protocol

class FileProcessor(Protocol):
    """Protocol for file processors."""

    def process(self, filepath: Path) -> bool:
        """Process the file and return True if successful, False otherwise."""
```

```

class PaperlessAPIProcessor:
    """Processor for uploading files to the Paperless API."""

    def __init__(self, vars: PaperlessVars) -> None:
        """Initialize with PaperlessVars."""
        self.vars = vars

    def process(self, filepath: Path) -> bool:
        """Process the file and upload it to the Paperless API."""
        url = f"{self.vars.api_url.rstrip('/')}{self.vars.api_path}"
        headers = {"Authorization": f"Token {self.vars.api_token}"}
        try:
            with filepath.open("rb") as f:
                response = requests.post(url, headers=headers, files={"document": f}, timeout=10)
        except requests.ConnectionError as e:
            logger.error(f"Failed to connect to Paperless API: {e}")
            return False
        except urllib3.exceptions.MaxRetryError as e:
            logger.error(f"Max retries exceeded while connecting to Paperless API: {e}")
            return False
        except urllib3.exceptions.NameResolutionError as e:
            logger.error(f"Name resolution error while connecting to Paperless API: {e}")
            return False
        logger.debug(f"Response from Paperless: {response.status_code} {response.text}")
        if response.status_code == 200:
            logger.info(f"Uploaded to Paperless: {filepath}")
            return True
        else:
            logger.error(f"Failed to upload {filepath} to Paperless: {response.status_code} {response.text}")
            return False

```

```

class EmailProcessor:
    """Processor for sending files via email to bookkeeping."""

    def __init__(self, vars: EmailVars) -> None:
        """Initialize with EmailVars."""
        self.vars = vars

    def process(self, filepath: Path) -> bool:
        """Process the file and send it via email."""
        msg = new_email_message(
            subject=filepath.name,
            smtp_to=self.vars.to,
        )
        msg.set_content(f"Hi,\n\nPlease find attached the file: {filepath.name}.\n\nBest regards.")
        with filepath.open("rb") as f:
            msg.add_attachment(
                f.read(),
                maintype="application",
                subtype="octet-stream",
                filename=filepath.name,
            )
        try:
            send_email(msg=msg)
        except Exception as e:
            logger.error(f"Failed to send {filepath} via email: {e}")
            return False
        else:
            logger.info(f"Sent successfully via email: {filepath}")
            return True

```

End result flow

1. Drop a pdf in the desired folder (phone / laptop)
2. It gets synced to dropbox and the NAS
3. Hourly the script runs and processes the files
 - Depending on the folder, multiple processors are run
4. When successful, the file is moved to the done folder

Once in a while I check the paperless inbox to process the docs

Take away

- Don't forget to learn while you create/automate
- Having uv makes this a breeze (especially, on a NAS)
- Awesome that something this easy, saves me hours
 - The image snapping bit
 - Dropping attachments from email as well
 - but also finding documents
 - Marking them correctly help enormously

What can you automate in your life?

- Bart Dorlandt
- <https://linkedin.com/in/bartdorlandt/>
- <https://dreamnetworking.nl/>
- https://github.com/bartdorlandt/paperless_email_processor
- 14:15 - Camera 5
 - Json freedom or chaos; how to trust your data

